

Goal Regression Planning for Blocksworld in Python

Diego Landaeta Torres - CSCE 489

Abstract

This project implements a goal-regression planner for Blocksworld in Python. The motivation for this project was following the courses: CSCE 420 and CSCE 489 from Texas A&M University, taught by Dr. Thomas Ioerger. The planner uses backward search with weakest-preimage regression. PDDL-style actions are parsed using the provided Sexpr.py parser and then converted into tuple-based literals and action representations (ActionSchema and GroundAction). Ground actions are generated over objects: A, B, C, and table. In order to keep the search manageable, the implementation uses relevance filtering, basic consistency checks, relaxed handling of temporary subgoal interference, and action ordering. Validation of candidate plans is done by forward simulation and a small cleanup step removes redundant adjacent actions. The system was tested on three presets: a simple stacking case, a destination-clearing scenario, and a case where the goal is already satisfied.

Introduction and Background

Classical AI planning studies how an agent can automatically construct a sequence of actions that transforms an initial world state into a desired goal state. In this project, the planning domain is Blocksworld, where blocks can be picked up and stacked under logical constraints (for example, a block must be clear before it can be moved). Blocksworld is widely used in AI courses because it is small enough to model clearly,

but still produces meaningful planning challenges such as action dependencies, subgoal ordering, and temporary goal interference.

The motivation for this project was to implement a working planner that directly reflects concepts from CSCE 420 and CSCE 489, especially backward search and goal regression. The focus was on building a correct and understandable regression-based planner over PDDL-style operators.

This implementation uses weakest-preimage regression as the core reasoning step. For goal set G and action a , regression is defined as:

$$\text{Regress}(G, a) = G \setminus \text{Add}(a) \cup \text{Precond}(a)$$

Approach and Implementation

I split the project into a few simple files so it was easier to build and debug. I kept the provided `Sexpr.py` parser unchanged and used it to read the PDDL-style actions from `blocksworld.PDDL`. Then in `pddl_parser.py`, I pulled out each action's parameters, preconditions, add effects, and delete effects.

I represented literals as tuples, like `("on","A","B")`, `("clear","A")`, and `("hand-empty",)`, because tuples are easy to compare and store in sets. I used two action classes: `ActionSchema` for the lifted action definitions, and `GroundAction` for fully instantiated actions.

For grounding, I used the objects `A`, `B`, `C`, and `table`. I filtered out obviously bad assignments, like trying to move `table` as a block or putting a block on itself.

The planner itself uses backward goal regression. It starts from the goal and works backward through relevant actions using the equation from the introduction.

Search is done with a depth-first stack over regressed goal sets. I also track visited goal sets to avoid repeating the same states.

To keep search from getting too big, I added simple controls: relevance filtering, lightweight action ordering, a small max depth, and a small branching cap per node. I also added consistency checks to skip impossible regressed goals, like hand-empty and holding(x) at the same time, holding two blocks at once, or a block being in two places.

At the end, I do a forward simulation check to make sure the final plan is actually executable from the initial state. I also run a small cleanup step to remove obvious adjacent redundant actions if removing them still keeps the goal satisfied.

Results

I tested the planner on three preset cases in main.py.

Preset 1 (simple stacking):

Initial state had A and B on the table, both clear, and hand-empty.

Goal was on(A, B).

The planner found the expected 2-step plan:

1. pickup A table
2. puton A B

So this confirmed the base regression flow worked for a straightforward case.

Preset 2 (destination-clearing scenario):

Initial state had on(B, C) and goal was on(A, C), so C had to be cleared first.

The planner found the expected 4-step plan:

1. pickup B C
2. puton B table
3. pickup A table
4. puton A C

This was the most important test because it checks subgoal interaction (you cannot place A on C until C is cleared).

Preset 3 (goal already satisfied):

Initial state already included on(A, B), and the goal was also on(A, B).

The planner correctly returned no actions needed.

Overall, the planner produced correct plans for all three target scenarios and handled both simple and interaction-based cases.

Challenges and Conclusion

The main difficulty in this project was balancing pruning in regression search.

At first, I pruned too aggressively, especially when an action deleted a current goal literal. That made the planner miss valid paths in interaction cases, like Preset 2 where one block has to be moved first to clear another block.

After relaxing that behavior, I hit the opposite issue: not pruning enough caused too many branches and very long traces. So the final version uses a middle ground with simple controls: relevance filtering, consistency checks, basic action ordering, and depth/branch limits.